UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación · Cátedra de Base de Datos II

Trabajo Práctico Integrador (TPI) — Informe Técnico de Segunda Entrega (Unidades 4 a 8)

Proyecto Integrador: Food Store
Comisión: C3
Docente: Prof. Sergio Neira
Equipo de Trabajo:

- Máximo Franco
- Andrés Novello
- Octavio Fiore

Fecha de Presentación: 12 de Noviembre de 2026
Ecosistema de Repositorios Oficiales:

- 🏛️ **Repositorio Central (PostgreSQL 16+ · Capa Relacional OLTP - Unidades 1 a 6):** https://github.com/MaxiFranco05/DB2
- 🍃📱 **Repositorio NoSQL & Móvil (MongoDB 7+ & SQLite 3.37+ STRICT - Unidades 7 y 8):** https://github.com/MaxiFranco05/DB2-NoSQL

Matriz de Cumplimiento y Trazabilidad de los 14 Objetivos

El presente trabajo acredita de manera formal y verificable la totalidad de los 14 objetivos exigidos por la cátedra para la segunda entrega parcial:

#	Unidad	Objetivo Acreditado	Archivo de Evidencia / Código	Resultado Verificado
1	**U4**	Refactorización de esquema a Forma Normal de Boyce-Codd (FNBC)	`sql/tp_fnbc_control_lote.sql`	Descomposición sin pérdida preservando dependencias funcionales reales.
2	**U4**	Migración de datos sin pérdida en transacción reversible con plan de reversión	`sql/tp_fnbc_control_lote.sql`	`BEGIN ... ROLLBACK;` y `EXCEPT` bidireccional = 0 filas de discrepancia.
3	**U4**	Desnormalización controlada documentada con salvaguardas	`sql/tp_desnormalizacion_top_categorias.sql`	Tabla `categoria_top_ventas` sincronizada automáticamente por trigger.
4	**U5**	Esquema de roles y permisos bajo principio de mínimo privilegio	`sql/roles.sql`	6 roles segregados (`rol_app_lectura` , `rol_app_escritura` , `rol_soporte` , `rol_reportes` , `app_web` , `admin_datos`).
5	**U5**	Auditoría y simulacro de acceso indebido reconstruido del log con contención	`sql/simulacro_incidente.sql` `docs/informe_incidente.md`	Escalamientos `GRANT` y `DROP TABLE auditoria` denegados y contenidos de inmediato.
6	**U5**	Auditoría de permisos asistida por IA con decisión humana documentada	`docs/DUIA/DUIA-TP-Unidad5.md`	Eliminación de contraseñas hardcodeadas en texto plano en favor de variables de entorno.
7	**U6**	Plan de mantenimiento integral automatizado con scripts y tareas rutinarias	`sql/maintenance.sql` `sql/health_check.sql`	Rutinas periódicas de `VACUUM ANALYZE` , reindexación y chequeo de salud del motor.
8	**U6**	Simulacro de desastre contra reloj con informe post-incidente	`docs/informe_u6.md` `docs/informe_u6.pdf`	**RTO observado: 6,95 segundos** (contra SLA < 300 s). Paridad de 891.992 registros (100%).
9	**U7**	Modelado documental en MongoDB con consultas sobre datos reales	`consultas_mongodb.js` `modelo_documental.md`	Carga de 5 pedidos con `Decimal128` , validador `$jsonSchema` y consultas Q-M1 a Q-M4.
10	**U7**	Comparación práctica y documentada entre solución documental y relacional	`consigna5_tabla_comparativa.md` `consultas_postgresql.sql`	Paridad simétrica Q-M2/Q-PG1 (2 filas) y Q-M3/Q-PG2 (4 filas) con precios congelados.
11	**U8**	Base SQLite especificada en Kiro y generada en modo STRICT	`.kiro/specs/tp8-tablet/` `movil/schema_sqlite.sql`	5 tablas STRICT con dinero en centavos enteros (`INTEGER`) e importación con `fecha_copia` .

#	Unidad	Objetivo Acreditado	Archivo de Evidencia / Código	Resultado Verificado
12	**U8**	Sincronización simulada contra servidor con resolución de conflictos	`movil/semaforo.sql` `movil/casos_simulacion.sql`	Batería 100% automatizada: C2 (`CONFLICTO` resuelto), C3 (`RECHAZADO`), C4 (lápida), C5 (`APROBADO`) y C6 (`UNIQUE`).
13	**U8**	Code review final asistido por IA de toda la cursada con comprobaciones	`docs/consigna_3c.md`	5 hallazgos auditados en PostgreSQL, MongoDB y SQLite con comprobaciones empíricas.
14	**Global**	Capa de datos completa con specs, scripts versionados, DUIA y guión oral	Repositorios GitHub `DB2` y `DB2-NoSQL`	Ecosistema probado de punta a punta y listo para su defensa oral.

---

1. Qué elementos se implementaron en cada unidad (4 a 8)

Unidad 4 · Normalización avanzada y redundancia

- **Refactorización a FNBC (`control_lote_almacen`):** Se abordó la extensión logística mayorista de *Food Store*. A partir de las dependencias funcionales $DF_1: {\text{LoteID}, \text{DepositoID}} \to \text{ResponsableControlID}$ y $DF_2: \text{ResponsableControlID} \to \text{DepositoID}$, se identificó que el determinante $\text{ResponsableControlID}$ en DF_2 no era superclave, violando la FNBC. Se descompuso el esquema en dos relaciones normalizadas:
  1. `responsable_deposito (responsable_id PK, deposito_id)`
  2. `lote_control (lote_id, responsable_id, PK (lote_id, responsable_id))`
- **Desnormalización controlada:** Se creó la tabla resumen `categoria_top_ventas` para acelerar el tablero directivo, evitando costosas operaciones de agregación periódicas (`SUM` , `JOIN`). Se implementó un trigger de sincronización reactiva que actualiza los totales ante inserciones en `detalle_pedido` .

Unidad 5 · Administración y seguridad

- **Esquema de Roles de Mínimo Privilegio (RBAC):** Creación de roles segregados sin superusuario:
  - `rol_app_lectura` : Solo `SELECT` en tablas operativas.
  - `rol_app_escritura` : `INSERT` , `UPDATE` controlados (sin `DELETE`).
  - `rol_soporte` : `SELECT` amplio con función de desbloqueo de usuarios.
  - `rol_reportes` : Acceso a vistas materializadas y agregaciones.
  - `app_web` : Rol operativo web con membresía en lectura y escritura.
  - `admin_datos` : DDL restringido y administración de esquemas.
- **Seguridad de Autenticación y Auditoría:**
  - Función `fn_autenticar(mail, password)` con hashing criptográfico seguro (`crypt` y `gen_salt('bf')` de `pgcrypto`).
  - Tabla central de `auditoria` activada mediante triggers que capturan timestamps, usuario, tipo de operación DML y valores anteriores/nuevos en formato JSON.

Unidad 6 · Mantenimiento y continuidad operativa

- **Plan de Mantenimiento:** Script transaccional `sql/maintenance.sql` y monitor de salud `sql/health_check.sql` para chequear tablas con alto bloat, índices no utilizados y estado de conexiones.
- **Rutina de Respaldo Automatizado:** Procedimiento en PowerShell que ejecuta `pg_dump -Fc` con compresión binaria y catálogo de objetos (TOC), verificando códigos de salida y generando logs fechados.

Unidad 7 · NoSQL y bases de datos documentales (MongoDB)

- **Modelado Documental e Inmutabilidad:** Migración de 5 pedidos de producción (`756504` a `756508`) del cliente `80584` hacia `foodstore_nosql.pedidos`, congelando precios históricos en el array embebido `items`.
- **Tipos de Alta Precisión (`Decimal128`):** Todos los importes monetarios (`total`, `precio_unitario_historico`, `subtotal`) se almacenan como `NumberDecimal` en BSON, eliminando cualquier imprecisión de coma flotante binaria.
- **Validador de Esquema (`$jsonSchema`):** Colección blindada mediante validador formal que exige tipos nativos, campos requeridos y garantiza la regla de integridad RI-M2 (array `items` con `minItems: 1`).
- **Consultas Documentales y Contraste de IA:** Implementación en `consultas_mongodb.js` de Q-M1 a Q-M4 y ejecución de prueba de laboratorio con rechazo de duplicados por índice único sobre `pedido_id_origen`.

Unidad 8 · Bases de datos móviles e integración (SQLite)

- **Esquema Móvil Desconectado con Modo STRICT:** Modelo embebido en SQLite con 5 tablas en modo estricto y dinero expresado en **centavos enteros (`INTEGER`)** para asegurar exactitud matemática total en ferias sin señal.
- **Instantánea con Marca Temporal (`fecha_copia`):** Exportación de catálogo mediante `v_snapshot_movil` en PostgreSQL con `to_char(CURRENT_TIMESTAMP, ...)` hacia `servidor_snapshot.csv`, importada a `snapshot_servidor` y transferida a `producto_cache`.
- **Algoritmo del Semáforo con Prioridades Rigurosas:** Evaluación jerárquica: Rechazo por stock o baja lógica (Prioridad 1) sobre Conflicto de precio (Prioridad 2) sobre Aprobación simulada (Prioridad 3).
- **Batería de Simulación 100% Automatizada:** Script `movil/casos_simulacion.sql` ejecutable de principio a fin demostrando la resolución de C2, quiebre de C3, lápida de C4, aprobación de C5 e idempotencia de C6 sin código comentado.

---

2. Cómo se probó su funcionamiento

PROTOCOLO DE PRUEBAS DE FOOD STORE	
1. Transacciones Reversibles	En U4 y U8 se encapsularon las migraciones y pruebas en bloques BEGIN ... ROLLBACK; verificando 0 filas residuales.
2. Copias de Prueba Aisladas	Ninguna prueba destructiva se corrió en producción: se usaron las bases food_store_copia y foodstore_restore_test.
3. Simulacro de Acceso Indebido	En U5 se ejecutó sql/simulacro_incidente.sql conectando con app_web para forzar login fallido, exfiltración y sabotaje.
4. Simulacro de Desastre Contra Reloj	En U6 se midió la restauración completa contra reloj desde dump binario con script cronometrado automatizado.
5. Batería de Conflictos en Sincronización	En U8 se simularon los casos C2 a C6 en SQLite evaluando cambio de precio, quiebre de stock, baja lógica y reenvíos.

1. **Transacciones Reversibles y Validación Matemática (U4):**
   La migración de datos hacia el nuevo esquema FNBC se ejecutó dentro de una transacción reversible. Antes del `COMMIT` final, se ejecutó una consulta de validación cruzada utilizando el operador relacional `EXCEPT` bidireccional entre la tabla origen y las tablas descompuestas, arrojando **exactamente 0 filas de diferencia** (cero pérdida de datos).
2. **Simulacro de Acceso Indebido y Sabotaje (U5):**
   Bajo la identidad del rol web `app_web`, se simularon 4 intentos fallidos de autenticación (fuerza bruta), seguidos de un login exitoso, la lectura de la tabla de usuarios y dos intentos de sabotaje:
   - Intento de escalamiento: `GRANT admin_datos TO rol_soporte;`
   - Intento de borrado de rastros: `DROP TABLE auditoria;`
     Ambos ataques fueron frustrados y rechazados de inmediato por el motor debido a la falta de privilegios DDL y ausencia del atributo `ADMIN OPTION`.
3. **Simulacro de Desastre y Recuperación (U6):**
   Se generó un respaldo en frío con `pg_dump -Fc` de la base de datos `foodstore` (891.992 registros totales). En un entorno limpio (`foodstore_restore_test`), se disparó la restauración automatizada cronometrando el tiempo de ejecución exacto desde la inicialización hasta la última verificación de restricciones lógicas.
4. **Verificación de Equivalencia Cruzada Relacional vs Documental (U7):**
   Se contrastaron los resultados de MongoDB frente a las consultas equivalentes en PostgreSQL (`consultas_postgresql.sql`):
   - Q-PG1 vs Q-M2: Retornaron idénticos 2 pedidos (`756505` y `756508`).
   - Q-PG2 vs Q-M3: Retornaron idénticos 4 pedidos (`756505` , `756506` , `756507` , `756508`).
   - Se comprobó la integridad exacta del precio histórico del producto 243882 en ambos motores ($3628,43).
5. **Batería de Pruebas del Semáforo Móvil (U8):**
   Se ejecutaron los casos de prueba sobre `foodstore_movil.db` mediante el script unificado `casos_simulacion.sql` :
   - **Caso C2 (Cambio de Precio):** La tablet tenía el Producto 1 a 421451 centavos y el servidor subió a 450000. El semáforo marcó `CONFLICTO` . Tras la aceptación del nuevo precio por parte del cliente, se generó una nueva solicitud (`req-uuid-c2-0002`) que pasó a `SIMULADO_APROBADO` .
   - **Caso C3 (Falta de Stock):** Solicitud de 50 unidades sobre 36 disponibles. Marcó `RECHAZADO` por stock insuficiente.

- **Caso C4 (Baja Lógica / Lápida):** Producto 3 desactivado en servidor (`activo = false`). Marcó `RECHAZADO` y conservó la fila en `producto_cache` con `disponible = 0` y `eliminado = 1` para no romper la integridad referencial de los borradores previos.
- **Caso C5 (Sin Cambios):** Coincidencia total en precio, stock y disponibilidad. Marcó `SIMULADO_APROBADO` .
- **Caso C6 (Idempotencia de Reintentos):** Reenvío del mismo UUID arrojó error `UNIQUE constraint failed: cola_sync.solicitud_uuid` , garantizando que ninguna transacción se procese por duplicado.

---

3. Qué resultados se obtuvieron

3.1. Métricas de Recuperación ante Desastres (Unidad 6)

El simulacro de recuperación ante desastres arrojó una performance excepcional, superando con creces los Acuerdos de Nivel de Servicio (SLA) fijados para la plataforma:

Métrica	Definición Operativa	SLA Previsto	Valor Real Observado	Cumplimiento
RTO (Recovery Time Objective)	Tiempo total de recuperación y verificación	< 300,00 s (5 min)	**6,95 segundos**	**-97,7% (Óptimo)**
RPO (Recovery Point Objective)	Pérdida máxima admisible de transacciones	$\le$ 24,00 horas (Diario)	**332,3 segundos** (~5,5 min)	**Cumplido**

Control de Integridad y Cardinalidad Post-Restauración (U6):

- `categoria` : 5 / 5 filas (100% coincidencia).
- `usuario` : 20.000 / 20.000 filas (100% coincidencia).
- `producto` : 50.000 / 50.000 filas (100% coincidencia).
- `pedido` : 200.000 / 200.000 filas (100% coincidencia).
- `detalle_pedido` : 621.987 / 621.987 filas (100% coincidencia).
- **Anomalías detectadas:** 0 productos con precio/stock negativo; 0 detalles con cantidad inválida.

3.2. Aislamiento Total "Móvil Propone, Servidor Dispone" (Unidad 8)

Para comprobar de forma rigurosa la consistencia de la arquitectura, se ejecutó en PostgreSQL (`consulta_final.sql`) el control posterior a la simulación móvil:

```
-- 1. Control de aislamiento de pedidos en central:
SELECT COUNT(*) AS pedidos_nuevos_en_servidor
FROM pedido
WHERE fecha >= '2026-11-05 18:00:00';

-- 2. Estado oficial del catálogo en PostgreSQL tras los cambios del servidor:
SELECT id, nombre, precio, stock, activo
FROM producto
WHERE id IN (1, 2, 3, 4)
ORDER BY id;
```

- **Resultados reales obtenidos en consola PostgreSQL:**

```
 pedidos_nuevos_en_servidor
----------------------------
                           0
(1 fila)

 id |   nombre   | precio  | stock | activo
----+------------+---------+-------+--------
  1 | Producto 1 | 4500.00 |   184 | t
  2 | Producto 2 |  766.89 |    36 | t
  3 | Producto 3 | 3289.26 |   199 | f
  4 | Producto 4 |  630.36 |   154 | t
(4 filas)
```

- **Coherencia técnica absoluta verificada:**
  1. En PostgreSQL, el `Producto 1` efectivamente refleja su nuevo precio oficial ($4500.00) y el `Producto 3` se encuentra dado de baja lógica (`activo = false`) debido a los eventos de servidor (`casos_servidor.sql`).
  2. Al mismo tiempo, el conteo de pedidos nuevos es `0` y los stocks (184, 36, 199, 154) permanecieron **100% inalterados**, acreditando que las solicitudes locales marcadas como `SIMULADO_APROBADO` en la tablet jamás alteran la base de producción sin sincronización formal previa.

---

4. Qué decisiones de diseño se tomaron y su justificación

1. Refactorización a FNBC vs. Desnormalización Controlada (U4)

- **Por qué FNBC en control de lotes:** La relación original contenía la dependencia transitiva $\text{ResponsableControlID} \to \text{DepositoID}$. Mantenerla provocaba anomalías de actualización (si un responsable cambiaba de depósito o se reasignaba, debían modificarse múltiples filas) y de inserción (no se podía dar de alta un responsable sin asignarle un lote previo). Descomponer a FNBC eliminó de raíz la redundancia y las anomalías.
- **Por qué desnormalización en categorías:** El cálculo de ventas totales por categoría requería escanear `detalle_pedido` (621.987 filas) con múltiples `JOIN` s en cada consulta del dashboard gerencial. Se aplicó desnormalización controlada persistiendo los acumulados en `categoria_top_ventas` , blindando la consistencia mediante triggers que recalculan exclusivamente la categoría afectada en tiempo de inserción.

2. Esquema de Roles y Seguridad (U5)

- **Principio de Mínimo Privilegio:** Se rechazó la práctica común de conectar la aplicación con un usuario administrador o superusuario. Se desacopló la lectura (`rol_app_lectura`) de la escritura (`rol_app_escritura`), impidiendo sentencias `DELETE` o DDL al rol web `app_web` .
- **Trazabilidad Forense:** Toda mutación sensible es auditada en `auditoria` guardando el estado pre y post mutación en JSONB, asegurando que un incidente de seguridad pueda reconstruirse sin ambigüedad a partir de los logs.

3. Selección Tecnológica: Almacén Documental (MongoDB) vs. Relacional (PostgreSQL) (U7)

- **Transacciones y Fuente de Verdad en PostgreSQL:** La compra activa, el control de concurrencia ACID y el descuento de stock físico deben residir indispensablemente en el motor relacional para prevenir condiciones de carrera y garantizar consistencia inmediata.
- **Historial Inmutable en MongoDB:** Una vez finalizado el pedido, la consulta frecuente del cliente no requiere transaccionalidad ni bloqueos. Almacenar el pedido en MongoDB con sus ítems y cliente embebidos permite

recuperarlo en una sola lectura de disco ($O(1)$) sin realizar ningún `JOIN`. Congelar `precio_unitario_historico` con `Decimal128` dentro del documento previene que cambios posteriores en la tabla de productos alteren las facturaciones históricas.

4. Persistencia Desconectada en SQLite con STRICT y Centavos (U8)

- **Tipificación Estricta (`STRICT`):** SQLite por defecto utiliza afinidad de tipos dinámica, lo que permite almacenar texto en columnas numéricas. Activar `STRICT` forzó la validación estricta de tipos nativos en el motor cliente.
- **Dinero en Centavos Enteros:** Se descartó el tipo `REAL` / `FLOAT` para evitar imprecisiones acumulativas derivadas de la aritmética de punto flotante binario (IEEE 754). Multiplicar por 100 y almacenar enteros garantiza exactitud contable absoluta.

---

5. Herramientas de IA utilizadas, prompts, descartes y DUIA

5.1. Matriz de Herramientas de IA Utilizadas

Herramienta	Rol y Finalidad en el Proyecto
Kiro (SDD)	Especificación dirigida por requisitos en notación EARS (`requirements.md`, `design.md`, `tasks.md`) para la arquitectura offline de U8.
Antigravity (LLM)	Generación de scripts DDL en modo `STRICT`, algoritmos transaccionales de sincronización (`semaforo.sql`) y estructuras de consultas MongoDB.
OpenCode (Agente Plan)	Auditoría técnica y code review integral de los scripts de la cursada (PostgreSQL, MongoDB y SQLite) con comprobaciones en base de datos.

5.2. Descartes Técnicos Fundamentados (Decisión Humana)

En estricto cumplimiento del protocolo de la cátedra, cada propuesta formulada por los asistentes de IA fue auditada por el equipo, **rechazando o modificando aquellas que comprometían la integridad, la seguridad o el alcance del proyecto**:

MATRIZ DE DESCARTES TÉCNICOS FUNDAMENTADOS

Unidad	Propuesta Formulada por la IA	Decisión	Justificación Técnica del Equipo (Máximo, Andrés, Octavio)
U4	Desnormalizar tablas maestras sin control de triggers	Descartado	Introducía redundancia sin garantías de consistencia.
U5	Contraseñas en texto plano en scripts SQL de roles	Descartado	Se corrigió parametrizando con variables de entorno seguras.
U6	Ejecutar VACUUM FULL durante la rutina de mantenimiento	Descartado	VACUUM FULL adquiere un bloqueo exclusivo total. Se sustituyó por VACUUM ANALYZE concurrente.
U7	Almacenar contraseñas del cliente en el documento Mongo	Descartado	Violaba el principio de mínimo privilegio y protección de datos
U7	Generar ObjectIds sintéticos puros sin pedido_id_origen	Descartado	Rompía la trazabilidad con la fuente de verdad relacional.
U8	Usar tipos REAL (flotantes) para montos en SQLite	Descartado	Provoca errores de redondeo. Se forzó INTEGER en centavos.
U8	Actualizar stock del servidor directamente desde la tablet	Descartado	Viola la regla de oro: "El móvil propone, el servidor dispone".
U8	Modificar trigger de stock con UPDATE agresivo bloqueante	Descartado	Interferiría con los scripts de carga masiva de la cátedra.

5.3. Declaración de Uso de IA (DUIA) Oficial

Campo	Completar
Herramienta	Kiro (SDD) & Antigravity
Spec o prompt utilizado	`.kiro/specs/tp8-tablet/` (`requirements.md` , `design.md` , `tasks.md`): "*Diseñar esquema SQLite STRICT con catálogo en centavos, cola_sync y semáforo de sincronización.*"
Qué generó	Especificación formal en notación EARS y scripts SQL DDL/DML para funcionamiento offline de tablets en ferias.
Qué se aceptó	95% de la arquitectura relacional y el algoritmo de evaluación por prioridades (Rechazo > Conflicto > Aprobado).
Qué se modificó o descartó, y por qué	Se descartó el tipo `REAL` en SQLite reemplazándolo por `INTEGER` (centavos) para evitar imprecisiones binarias en dinero; se descartó cualquier mutación directa de stock desde el móvil.
Verificación realizada	✅ Pruebas de atomicidad (COMMIT/ROLLBACK) en SQLite. ✅ Ejecución de casos C2 a C6 en `foodstore_movil.db` . ✅ EXCEPT bidireccional = 0 filas de discrepancia en servidor central PostgreSQL.

Campo	Completar
Herramienta	OpenCode (Agente: Plan)
Spec o prompt utilizado	*"Auditoría integral y code review de scripts de PostgreSQL (U1-U6), MongoDB (U7) y SQLite (U8) detectando riesgos de concurrencia y seguridad con comprobaciones en base."*
Qué generó	Matriz de 5 hallazgos técnicos (roles con contraseñas en texto plano, validador `$jsonSchema`, carrera de stock pedagógica, tipos dinámicos SQLite, e idempotencia por UUID).
Qué se aceptó	80% de las propuestas (roles parametrizados, tipos estrictos, deduplicación e índices).
Qué se modificó o descartó, y por qué	Se descartó con fundamento el trigger de stock con `UPDATE` bloqueante para no romper la compatibilidad con los scripts de carga masiva de la cátedra (`Genera_registros.sql`).
Verificación realizada	✅ Comprobaciones reales en consola para cada hallazgo documentadas en `docs/consigna_3c.md` .

---

6. Documentación Complementaria y Anexos

Para una profundización técnica paso a paso de cada subsistema, se anexan las siguientes piezas de ingeniería:

1. **Anexo Técnico de Persistencia Móvil y SQLite (TP 8):** Documento monográfico de 28 páginas con el detalle exhaustivo de la implementación de la tablet, especificaciones EARS y casos de prueba (`food-store/docs/informe_u8.pdf`).
2. **Informe de Recuperación y Respaldo (Unidad 6):** Reporte post-incidente y análisis de RTO/RPO (`docs/informe_u6.pdf`).
3. **Informe de Modelado Documental (Unidad 7):** Matriz de consultas comparativas y paridad relacional (`informe_u7.pdf`).